

Computer Organization

Assignment 1: Answer all the following questions.

Submission: Submit a word or pdf file online in Brightspace.

Q1. What is the difference between computer organization and architecture?

Computer Architecture:

Computer architecture refers to the attributes of a computer system that are **visible to the programmer** and that have a **direct impact on the logical execution** of a program. It includes the **Instruction Set Architecture (ISA)** — instruction formats, opcodes, registers, data types, memory addressing techniques, and the effect of instructions on registers and memory.

Computer Organization:

Computer organization refers to the **operational units** and their **interconnections** that **implement** the architectural specifications. This includes hardware details such as the ALU, control signals, data paths, memory hierarchy, and how the CPU is internally structured. How those reg connected or are hidden to programmer is organisation and visible –

Q2. What are the four basic functions that a computer can perform?

1. Data Processing

Manipulating data to produce meaningful information.
Examples: arithmetic operations, logical operations.

2. Data Storage

Storing data for later use — either temporarily (RAM) or permanently (disk, SSD).

3. Data Movement (Input/Output)

Transferring data between the computer and the outside world.
Examples: keyboard input, monitor output, network communication.

4. Control

Directing and coordinating all other operations.
The control unit fetches instructions, interprets them, and issues the necessary control signals.

Final:-> which of these be the fastest ?

Data processing why?

Data Processing is the fastest because it occurs entirely **inside the CPU using registers and ALU**, avoiding slower memory or I/O delays.

Data storage->

Asked .Do we send control signal withing the processor ?

Depende where we sending the signals is to main memory or to hrd disk or inside

Or we say its all getting via ALU so it be as fast as processing as it is inside the cpu we have gates inside like send data or send control be the equally

Control signal ->?? Which is faster (internal or external)

'moving internally ram to internal faster

PR moving to outside like cd, disk be slower than data movement

Q3. Draw a top-level structure diagram of the computer representing the following components: I/O, Main Memory, System Bus, CPU, Registers, ALU, Internal Bus, Control Unit, Sequencing Logic, Control Unit Registers and Decoders, Control Memory.

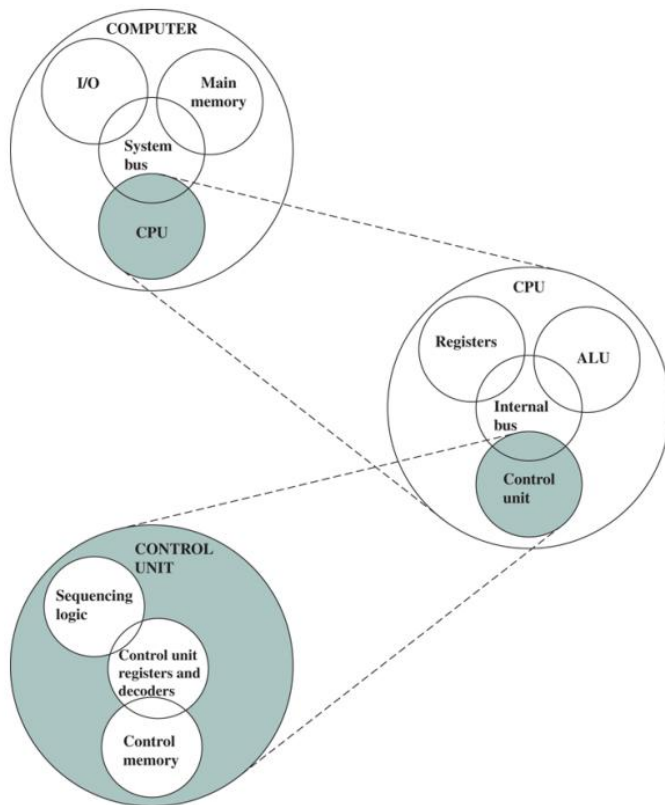


Figure 1.1 The Computer: Top-Level Structure

Q4. What is the difference between simple single processor computer and multicore computer structure?

A simple single-processor computer has only one CPU core that executes one instruction stream at a time. A multicore computer contains multiple CPU cores on the same chip, allowing parallel execution of multiple instructions and higher performance.

Single -> one unit

Multi -> multi unit more core

Extra IMP How two core communicate to each other. (**L3** Shared memory)

Q5. What are major elements of a multicore computer? Define and explain how are they related to each other?

A multicore computer contains multiple CPU cores on the same chip, each with private L1/L2 caches, and all cores share an L3 cache, main memory, I/O subsystem, and an interconnect bus. A coherence mechanism maintains data consistency between cores when they access shared memory. All these components work together to enable parallel processing and faster system performance.

Commented [B01]: Internal connection

Ex-Imp What is cache coherence system? – if you change in one location it will update all across (vertical and horizontal coherence)

Q6. Using an IAS memory format, what is the maximum possible data that can be stored? You can represent the answer as a power of 2.

2^{39} (IAS has one bit sign bit and rest 2^{39})

Q7. Let's say you want to add two registers AX and BX. Which of the following will be impacted when the processor executes the instruction. PC, MAR, MRO, MBR, IR

Let's analyze each:

◆ **1. Program Counter (PC) — YES, impacted**

- PC always updates to point to the **next instruction**. (for sure)
- After fetching ADD AX, BX, PC increments.

✓ Impacted

◆ **2. Memory Address Register (MAR) — NO**

- MAR holds an address when CPU accesses memory.
- This instruction **uses registers only**, no memory access. We are not getting any change here

Not impacted

◆ **3. Memory Buffer Register (MBR or MDR) — NO**

- Used only when reading/writing data between CPU and memory.
- Since both operands (AX, BX) are CPU registers, no memory fetch.

Not impacted

◆ 4. Instruction Register (IR) — YES, impacted

- IR must load the instruction ADD AX, BX after the fetch cycle.
- So IR definitely changes.

✓ Impacted

◆ 5. Memory Read out register (MRO) — NO

- No memory read is required (operands are inside CPU registers).

Not impacted

Final Answer (Exam-ready):

Impacted:

- PC
- IR

NOT impacted:

- MAR
- MBR/MDR
- MRO

Q8. What are the two general approaches to developing an embedded operating system?

There are **two general approaches** to developing an embedded operating system:

1. Adapt an existing operating system

An existing OS (such as embedded Linux, Windows CE, or other commercial/proprietary systems) is **modified** to suit the embedded application.

2. Design a dedicated embedded operating system

A new OS is **designed and implemented specifically for embedded use**.
Example: **TinyOS**, widely used in wireless sensor networks.

Embedded Operating Systems

There are two general approaches to developing an embedded operating system (OS). The first approach is to take an existing OS and adapt it for the embedded application. For example, there are embedded versions of Linux, Windows, and Mac, as well as other commercial and proprietary operating systems specialized for embedded systems. The other approach is to design and implement an OS intended solely for embedded use. An example of the latter is TinyOS, widely used in wireless sensor networks. This topic is explored in depth in [STAL18].

Which one will be faster – designing from scratch

Q9. Define and explain the techniques used in contemporary processors to achieve speed up?

Pipelining

Definition:

Pipelining divides instruction execution into multiple stages (Fetch, Decode, Execute, Memory, Write-back).

How **it** **increases** **speed:**
Multiple instructions are processed simultaneously in different stages, just like an assembly line.

Result:

Increases *instruction throughput* (more instructions completed per unit time).

2. Superscalar Architecture

Definition:

A superscalar CPU issues **multiple instructions per clock cycle** by using several execution units.

How **it** **increases** **speed:**
Parallel execution → more work done per cycle.

Result:

Improves IPC (Instructions Per Cycle).

3. Out-of-Order Execution (OoO)

Definition:

CPU executes instructions **as soon as operands are available**, not strictly in program order.

How	it	increases	speed:
Avoids waiting on slow instructions (like a cache miss).			
Fills empty CPU cycles with independent instructions.			

4. Branch Prediction

Definition:

CPU guesses the outcome of **conditional branches** (if-else, loops) before they are resolved.

How	it	increases	speed:
Good predictions keep the pipeline full.			
Bad predictions cause pipeline flushing (slowdown).			

Result:

High prediction accuracy $\Rightarrow$ major speed-up.

more elaborate techniques for feeding the monster. Among the techniques built into contemporary processors are the following:

- **Pipelining:** The execution of an instruction involves multiple stages of operation, including fetching the instruction, decoding the opcode, fetching operands, performing a calculation, and so on. Pipelining enables a processor to work simultaneously on multiple instructions by performing a different phase for each of the multiple instructions at the same time. The processor overlaps operations by moving data or instructions into a conceptual pipe with all stages of the pipe processing simultaneously. For example, while one instruction is being executed, the computer is decoding the next instruction. This is the same principle as seen in an assembly line.
- **Branch prediction:** The processor looks ahead in the instruction code fetched from memory and predicts which branches or groups of instructions are likely to be processed next. If the processor guesses right most of the time, it can prefetch the correct instructions and buffer them so that the processor is kept busy. The more sophisticated examples of this strategy predict not just the next branch but multiple branches ahead. Thus, branch prediction potentially increases the amount of work available for the processor to execute.
- **Superscalar execution:** This is the ability to issue more than one instruction in every processor clock cycle. In effect, multiple parallel pipelines are used.
- **Data flow analysis:** The processor analyzes which instructions are dependent on each other's results, or data, to create an optimized schedule of instructions. In fact, instructions are scheduled to be executed when ready, independent of the original program order. This prevents unnecessary delay.
- **Speculative execution:** Using branch prediction and data flow analysis, some processors speculatively execute instructions ahead of their actual appearance in the program execution, holding the results in temporary locations. This enables the processor to keep its execution engines as busy as possible by executing instructions that are likely to be needed.

These and other sophisticated techniques are made necessary by the sheer power of the processor. Collectively they make it possible to execute many instructions per processor cycle, rather than to take many cycles per instruction.

Quadruple

What is > Gold > Silver

What is drawback of speculative execution (Val change in variable and it system will change it and it will become wrong)

Q10. What changes have been introduced to balance the performance of memory with increasing microprocessor speeds?

As microprocessor speeds increased much faster than memory speeds, several architectural changes were introduced to reduce the “memory wall” problem. The most important improvements are:

- ✓ 1. **Cache Memory Hierarchy (L1, L2, L3):**
Fast caches store frequently used data close to the CPU, reducing slow RAM accesses.
- ✓ 2. **Memory Prefetch Interleaving:**
Main memory is divided into multiple banks that can be accessed in parallel, increasing overall bandwidth.
- ✓ 3. **Faster and Wider Memory Technologies:**
Development of DDR/DDR5 RAM, wider buses, and higher clock speeds improves data transfer rates. (DRAM)
4. **On-Chip Memory Controllers:**
Integrating the memory controller inside the CPU reduces latency and speeds up communication with RAM.

Which one will faster and why just search it and why DDR3 and DDR5

Q11. Discuss the challenges of Power, RC Delay and Memory Latency?

Modern processors face three main challenges: **power**, **RC delay**, and **memory latency**. As processors get **faster**, they use more power and produce more **heat**, which makes it difficult to keep increasing **speed** safely. RC delay happens because the tiny wires inside the chip become thinner as **technology shrinks**, which slows down the movement of signals inside the processor. Memory latency (**Time it takes to read something from the memory**) is another problem because the CPU is very fast, but the main memory (RAM) is much slower, so the processor often has to wait for data, reducing overall performance. These three issues make it harder to improve processor speed and efficiency.

Q12. What is the difference between Multicore Processors and Graphics Processing Units?

Multicore processors are CPUs that contain several general-purpose cores designed to run normal computer tasks like operating systems, applications, and multiple programs at the same time. They focus on fast decision-making, logic operations, and running a few complex tasks efficiently. In contrast, **Graphics Processing Units (GPUs)** have hundreds or thousands of small cores designed mainly for handling graphics, images, and large amounts of simple, repeated calculations in parallel. CPUs are good for complex tasks with a few threads, while GPUs are good for highly parallel work like gaming, AI, and scientific simulations

Q13. Explain Amdahl's law and little's law along with example calculations?

Amdahl's Law states that the maximum speedup of a program using multiple processors is limited by the fraction of the program that must run sequentially.

It shows that even with infinite processors, the speedup cannot exceed $1/(1 - P)$, where P is the parallelizable portion of the program.

In any program two part one in parallel one not

If 100 processor run same time but they not parallelised $\rightarrow$ speed up possible function which can run in parallel

Little's Law — Book Definition

Little's Law states that in a stable system, the average number of items in the system (L) is equal to the arrival rate (λ) multiplied by the average waiting time (W).

Mathematically:

$$L = \lambda \times W$$

- **Amdahl's Law:** Speedup from multiple processors is limited by the serial part of the program.
- **Little's Law:** Items in system = arrival rate $\times$ waiting time.

Q14. Why hardwired software is more efficient than general purpose computers running a software?

Hardwired systems are built using **fixed, dedicated electronic circuits** to perform a specific task. Because the hardware is designed only for that one function, it can execute operations **much faster**, with **less overhead** and **lower power consumption**. In general-purpose computers, the CPU must fetch instructions, decode them, and run them through several

layers of software, which takes extra time. Hardwired control directly performs the operation without this instruction-processing cycle. Therefore, hardwired systems are **faster, simpler, and more efficient** for a single dedicated task, while general-purpose computers trade speed for flexibility.

Q15. What are the four possible actions a processor can perform?

Four possible actions a processor can perform

A processor can perform the following four basic actions:

1. **Fetch** – It reads an instruction or data from memory.
2. **Decode** – It interprets or understands the fetched instruction.
3. **Execute** – It performs the operation (arithmetic, logic, or control action).
4. **Store/Write back** – It writes the result back to a register or memory.

Q16. What are the four classes of interrupts? Define and explain.

1. Program Interrupts

These occur when the processor detects an error or exceptional condition while executing an instruction. Examples include arithmetic overflow, division by zero, invalid opcode, or memory access violations.

2. Timer Interrupts

These are generated by an internal timer within the processor. They occur at regular intervals and allow the operating system to perform functions such as process scheduling and timekeeping.

3. I/O Interrupts

These interrupts come from input/output controllers to signal the completion of an I/O operation or request CPU service. Examples include disk read complete, keyboard input, or network packet arrival.

4. Hardware Failure Interrupts

These interrupts indicate a failure in hardware components. Examples include memory parity errors, power failure signals, or device malfunction

5 **Timer Interrupt** :-> if a code has for loop how it know to stop and so there is timer in processor if it goes beyond so that processor should not be work for long it should stop

Q17. Draw the Instruction cycle diagram with interrupts?

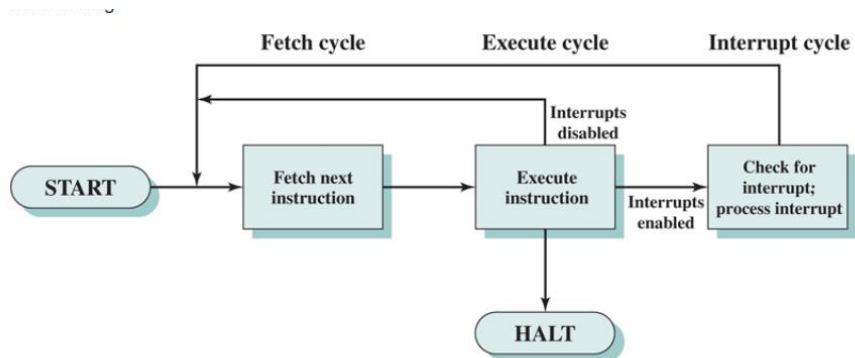


Figure 3.9 Instruction Cycle with Interrupts

Q18. What is the difference between sequential interrupt processing and nested interrupt processing?

Difference Between Sequential Interrupt Processing and Nested Interrupt Processing

Sequential Interrupt Processing

- In sequential interrupt processing, **only one interrupt is handled at a time**.
- When an interrupt occurs, the CPU disables all other interrupts until the current interrupt service routine (ISR) is finished.
- Other interrupts must **wait** in a queue.
- This method is simple but can cause delays if a high-priority interrupt must wait for a low-priority one to complete.

Nested Interrupt Processing

- In nested interrupt processing, the CPU **allows higher-priority interrupts to interrupt** a lower-priority interrupt handler.
- When a new, higher-priority interrupt occurs, the processor **pauses** the current ISR, handles the new one, and then returns to the previous ISR.
- This allows **faster response** to important interrupts.
- It requires hardware support and more complex interrupt handling.

Q19. Why is disabled interrupt approach in handling interrupts risky? What challenges does it pose using this method to handle multiple interrupts?

Disabling interrupts ensures the current ISR runs safely but is risky because it can block all interrupts, miss events, and cause potential system failures while handling multiple interrupts becomes inefficient.

Q20. Why L1 and L2 cache faster than L3 cache?

L1 and L2 caches are faster than L3 because they are closer to the CPU, smaller in size, and simpler in design, whereas L3 is larger, shared, and farther from the processor, making it slower to access.

Q21. In most computers L1 and L2 cache are faster than L3 cache, then what is the purpose of having a slow cache memory?

L3 cache is slower but larger and shared, helping the CPU avoid frequent main memory accesses and improving overall system efficiency.

Q22. What are three types of bus lines that are used to connect CPU, Memory and I/O devices?

- **Data bus:** carries data
- **Address bus:** carries addresses
- **Control bus:** carries control signals

Q23. If the processor wants to write something in memory, what will be transferred on the various bus lines?

When the **CPU wants to write data to memory**, the following happens on the bus lines:

- **Address Bus:** The CPU sends the **address** of the memory location where the data will be stored.
- **Data Bus:** The CPU sends the **actual data** to be written to that memory location.
- **Control Bus:** The CPU activates the **Write signal** to indicate a write operation, and may also use signals like **Memory Enable** or **Chip Select** to activate the correct memory module.

Q24. Point-to-point interconnection method is the faster way to connect various components. Explain the disadvantages of this method?

Disadvantages of Point-to-Point Interconnection

Although **point-to-point interconnection** provides **fast and direct communication** between components, it has several disadvantages:

- **High Cost:**
 - Each device requires a dedicated connection to every other device, which **increases hardware cost** significantly.
- **Complex Wiring:**
 - The number of wires grows rapidly with the number of devices, leading to **complex and bulky wiring** that is difficult to manage.
- **Scalability Issues:**
 - Adding a new device requires **new dedicated connections** to all other devices, making the system **difficult to expand**.
- **Limited Flexibility:**
 - Any change in the system configuration requires **rewiring or redesigning** the connections.

Q25. What is the difference between the link layer and routing layer in QPI four-layer architecture?

In the **QPI four-layer architecture**, the **Link Layer** ensures **reliable data transfer between two directly connected devices** by handling error detection, framing, and flow control. In contrast, the **Routing Layer** is responsible for **forwarding packets across multiple nodes**.

in the QPI network, determining the best path and managing addressing and switching. Simply put, the Link Layer focuses on **data integrity on a single link**, while the Routing Layer focuses on **data delivery across the entire network**.

Q26. What is the purpose of a switch in a typical PCIe configuration?

Purpose of a Switch in a PCIe Configuration

In a typical **PCIe (Peripheral Component Interconnect Express)** system, a **switch** is used to **expand the number of PCIe devices** that can be connected to the processor or root complex.

Functions of a PCIe Switch:

- **Multiple Device Support:**
 - Allows one PCIe root port to connect to **several endpoints** (graphics cards, SSDs, network cards) simultaneously.
- **Packet Routing:**
 - Receives packets from the root complex and **forwards them to the correct downstream device**.
- **Bandwidth Management:**
 - Shares the available PCIe lanes efficiently among multiple devices.
- **Transparency:**
 - Devices connected through the switch **appear as if directly connected** to the root complex.

Summary:

A PCIe switch **increases connectivity, manages traffic**, and **enables multiple devices** to communicate efficiently over PCIe lanes.

Q27. What is the role of the transaction layer in PCIe layered architecture?

Functions of the Transaction Layer:

The Transaction Layer acts as the **interface between software/CPU and the lower PCIe layers**, handling **packet formation, delivery, and flow control** to ensure reliable data communication across PCIe links.

- **Packet Creation:**

- Converts read, write, and message requests from software or hardware into TLPs.

➤ **Packet Processing:**

- Receives TLPs from other devices and **delivers them to the appropriate requester.**

➤ **Flow Control and Ordering:**

- Ensures **proper sequencing** of packets and that the receiving device can handle them without overflow.

Q28. Let's say you must write a computer program that is going to use 4 variables. Based on the principle of locality, where would you declare these variables?

Based on the principle of locality, the 4 variables should be declared as **local variables on the stack or in CPU registers**, because these locations provide **fast access** and take advantage of **temporal and spatial locality**.

Q29. Studies have shown that the data address probability distribution is wider than the instruction address probability distribution, what does this mean?

Studies show that the **data address probability distribution is wider than the instruction address distribution**, which means that programs tend to execute a **small set of instructions repeatedly** (narrow distribution), while **data accesses are spread over many memory locations** (wide distribution). As a result, **instruction caches** can be smaller and still effective, whereas **data caches** need to be larger or more sophisticated to handle the wider range of data addresses efficiently.

Q30. With the recent advances in big data, is spatial locality more important than temporal locality? Explain your answer.

Spatial locality has become **more important in big data applications** because large datasets are often processed sequentially or in blocks. Accessing contiguous memory regions allows **efficient prefetching** and better cache utilization. Temporal locality is still useful, but in big data, the sheer volume of data makes **predictable sequential access (spatial locality)** more critical for performance.

Q31. In a program, explain why there are more dynamic instructions than static instructions?

- **Static instructions** are the unique instructions written in the program code.
- **Dynamic instructions** are the actual instructions executed at runtime. Loops, function calls, and repeated operations cause the same static instructions to execute **many times**, resulting in **more dynamic instructions** than static instructions.

Q32. List and describe the key characteristics of a memory system.

- **Capacity:** Amount of data it can store.
- **Access Time / Latency:** Time taken to read/write a value.
- **Transfer Rate / Bandwidth:** Speed of data transfer between memory and CPU.
- **Cost per Bit:** Determines economic feasibility.
- **Volatility:** Whether memory retains data without power.
- **Physical Proximity to CPU:** Affects speed and latency.
- **Organization:** Cache, main memory, registers, etc., affects performance.

Q33. Why is associative access method more complex to design?

Associative (or fully associative) memory allows **any memory block to be placed in any location**, requiring the hardware to **compare the input address with all stored addresses simultaneously**. This **parallel comparison logic** is complex, costly, and consumes more power, making the design challenging.

Q34. What is transfer rate for a memory system and why is the calculation method different for random access and non-random access methods?

- **Transfer Rate:** Amount of data transferred per unit time (e.g., MB/s).
- **Random Access Memory (RAM):** Access time is **uniform**, so transfer rate is calculated as **data size ÷ access time**.
- **Non-Random Access Memory (e.g., tapes, disks):** Access time depends on **seek, rotation, and latency**, so transfer rate considers **average access time plus data transfer time**.

Q35. Arrange the following memory devices into memory hierarchy from closer to the processor to further away. Registers, On-chip cache, Off-chip cache, main memory, Hard disks, Magnetic Disks, Magnetic Tapes.

Registers → On-chip cache → Off-chip cache → Main memory → Hard disks → Magnetic disks → Magnetic tapes

Q36. Why L1 cache is smaller than L2 cache, L2 cache is smaller than L3 cache and so on?

- **Proximity vs Size Trade-off:** Smaller caches are faster and placed closer to the CPU (L1) for quick access.
- **Larger caches** (L2, L3) store more data but are slower and placed farther from the CPU.
- This balances **speed, cost, and capacity** in a memory hierarchy.

Q37. What is the design principle of inclusion in memory systems?

- **Inclusion Principle:** Data in a **higher-level cache (L1)** is also present in **lower-level caches (L2, L3)**.
- Ensures **coherence**, simplifies **cache management**, and makes it easier to **invalidate blocks** in lower caches when needed.

Q38. Which memory devices use vertical coherence, and which ones use hierarchical coherence?

- **Vertical Coherence:** Maintains consistency between **different levels of cache and main memory** (e.g., $L1 \leftrightarrow L2 \leftrightarrow \text{main memory}$).
- **Hierarchical Coherence:** Maintains consistency **within caches at the same level** across multiple processors (e.g., L1 caches in a multi-core CPU).

Q39. How is the memory access time calculated if there are two levels of cache used? Write the mathematical formula?

To express the average time to access an item, we must consider not only the speeds of the two levels of memory, but also the probability that a given reference can be found in M1. We have

$$\begin{aligned} T_s &= H \times T_1 + (1 - H) \times (T_1 + T_2) \\ &= T_1 + (1 - H) \times T_2 \end{aligned} \quad (4.2)$$

where

$$\begin{aligned} T_s &= \text{average (system) access time} \\ T_1 &= \text{access time of M1 (e.g., cache)} \\ T_2 &= \text{access time of M2 (e.g., main memory)} \\ H &= \text{hit ratio (fraction of time reference is found in M1)} \end{aligned}$$

Q40. Instead of using smaller l1 and l2 cache, why can't we use a bigger cache of size l1+l2?

Q40. Instead of using smaller L1 and L2 caches, why can't we use a bigger cache of size L1 + L2?

- **Trade-off between speed and size:** Large cache is **slower** due to longer access and lookup times.
- **Hierarchy advantage:** L1 provides **very fast access** to frequently used data, L2 stores less frequently used data, balancing **speed and capacity**.
- A single large cache would **increase latency** for most accesses, reducing CPU performance.